Parcial 2 — Programación III

Programación funcional con Elixir: recursividad, Streams y structs

Universidad del Quindío
Programa de Ingeniería de Sistemas y Computación
Docente: Julián Esteban Gutiérrez Posada

Problemas desarrollados: 1. Rutas de entrega en una red dirigida · 2. Identificación de regiones en un mapa de colores y exportación SVG

Integrantes: Sebastian Cortes Arango

Repositorio de GitHub: `https://github.com/IngScortes/Parcial_II`

Noviembre de 2026

Contenido

1. Repositorio y ejecución
2. Problema 1 — búsqueda, validación recursiva, Stream infinito y pruebas
3. Problema 2 — regiones, validación recursiva, SVG y pruebas
4. Reflexión sobre el uso de inteligencia artificial

1. Repositorio y ejecución

El código, los archivos de entrada utilizados, las salidas generadas y las instrucciones de ejecución están en el repositorio: `https://github.com/IngScortes/Parcial_II`

Las combinaciones válidas exigían un problema de Streams infinitos (1 ó 4) y uno de structs (1 ó 2). Se eligieron los problemas **1 y 2**: el 1 cubre Streams infinitos y structs (`Tramo`, `Itinerario`, `Consulta`) y el 2 cubre structs (`Color`, `Region`, `Mapa`).

Todo está escrito en scripts `.exs` (Elixir 1.14, sin dependencias ni `mix`). Cada módulo vive en su propio archivo dentro de `lib/`, y cada archivo carga sus dependencias con `Code.require_file`.

Diseño: separación del qué y el cómo

`Rutas` y `Mapas` son los orquestadores: su `main/0` es un pipeline que se lee como la especificación del problema (`argumentos |> ejecutar() |> generar_mensaje() |> Consola.mostrar()`) y `ejecutar` devuelve tuplas `{:ok, ...}` o `{:error, ...}`. Los algoritmos están en módulos separados (`Depuracion`, `Busqueda`, `Alcance`, `Validacion`, `Buscador`, `Svg`...), la presentación en `Reporte` (funciones puras) y la entrada/salida en `Lector`, `Escritor` y `Consola`. Las constantes son atributos de módulo y el código sigue la estructura de los ejemplos de clase.

Estructura del repositorio

```
problema1/  problema1.exs  pruebas.exs  lib/ (13 módulos, uno por archivo)
            datos/ datos_invalidos/ datos_limite/ datos_vacio/ datos_grande/   + salida*/
problema2/  problema2.exs  pruebas.exs  lib/ (11 módulos, uno por archivo)
            datos/ datos_invalidos/ datos_limite_fondo/ datos_limite_1x1/
            datos_limite_mapa_vacio/ datos_escape/ datos_grande/              + salida*/
informe/    generar_informe.py  evidencias/ (consola de cada caso)
ejecutar_todo.bat / ejecutar_todo.sh
```

Cómo ejecutar (desde la carpeta de cada problema)

```
elixir problema1.exs [carpeta_entrada] [carpeta_salida]    # por defecto: datos  ->  salida
elixir problema2.exs [carpeta_entrada] [carpeta_salida]
elixir pruebas.exs                                          # pruebas automáticas (ExUnit)
```

Cumplimiento de los requisitos generales

Requisito	Cómo se cumple
Lectura en flujo	Todos los archivos se leen con `File.stream!/1` (módulo `Lector`); nunca `File.read!/1` en el código.
Transformaciones perezosas	Limpieza, numeración de líneas y conversión con `Stream.with_index/map/reject/flat_map`; las listas se construyen después.
Comentarios y líneas en blanco	Se descartan las vacías y las que empiezan con #, conservando el número de línea física.
Salidas por flujo	`Stream.into(File.stream!(ruta))` o escritura incremental; el SVG se escribe por fragmentos (encabezado, rectángulos, cierre).
Errores de entrada	Archivo faltante: se informa cuál y se devuelve `{:error, ...}` sin excepción. Registros inválidos: línea y motivo.
Validación recursiva	`Depuracion` (P1) y `Validacion` (P2): recursión propia con acumulador; sin `Enum` ni `Stream` dentro.
Convenciones	Coordenadas desde 0; CSV con `;`.

2. Problema 1 — Rutas de entrega en una red dirigida

El programa lee los tramos con `Tramo.parse/1`, depura la red de forma recursiva, arma el grafo, y para cada consulta calcula (a) los centros alcanzables con un **Stream infinito** y (b) si el destino es alcanzable, todos los itinerarios simples por backtracking recursivo, con los óptimos por número de tramos, tiempo y costo (conservando empates).

2.1 Búsqueda de itinerarios (procesamiento principal)

Caso	Tipo	Entrada	Resultado obtenido
Ejemplo del enunciado	Válido	`datos/`	A→D: 4 itinerarios; menos tramos: 1, 3; menor tiempo: 1; menor costo: 3. B→A: sin itinerarios. Idéntico al enunciado.
Líneas, centros y consulta erróneos	Inválido	`datos_invalidos/`	Líneas 3 y 4 de tramos reportadas con motivo; centros 'C' y 'Z' inexistentes (no se inicia la búsqueda); consulta mal formada reportada. Sin excepción.
Carpeta sin archivos	Inválido	`carpeta_inexistente/`	Se informa cuáles archivos faltan y termina sin excepción.
Origen sin salidas; A→A; D→D	Límite	`datos_limite/`	D→A: sin itinerarios; A→A y D→D: itinerario trivial de 0 tramos.
Archivos vacíos	Límite	`datos_vacio/`	0 tramos, 0 consultas; termina correctamente y crea las salidas.
10 rombos en serie	Tamaño	`datos_grande/`	N0→N10: 1024 itinerarios (2^{10}), ejecución completa en menos de 1 s.

Caso válido

Entrada: problema1/datos/tramos.csv

```
# origen,destino,minutos,costo
A,B,8,5
B,D,7,4
A,C,6,2
C,D,12,1
B,C,3,2
C,B,4,1
A,A,2,1
A,B,9,7
```

Entrada: problema1/datos/consultas.txt

```
A,D
B,A
```

Salida: problema1/salida/itinerarios.txt

```
=== A → D ===
Itinerario 1: A → B → D | tramos: 2 | 15 min | costo: 9
Itinerario 2: A → B → C → D | tramos: 3 | 23 min | costo: 8
Itinerario 3: A → C → D | tramos: 2 | 18 min | costo: 3
Itinerario 4: A → C → B → D | tramos: 3 | 17 min | costo: 7
Total: 4 itinerarios
Menos tramos: 1, 3
Menor tiempo: 1
Menor costo: 3

=== B → A ===
sin itinerarios
```

Consola

```
Tramos bien formados: 8
Tramos descartados en la depuración: 2

Resumen por consulta:
  A → D: 4 itinerarios
  B → A: sin itinerarios
```

Caso inválido

Entrada: problema1/datos_invalidos/tramos.csv

```
# casos inválidos y límite
A,B,8,5
B,C,x,2
C,D
A,B,9,7
D,D,1,1
B,A,2,2
```

Entrada: problema1/datos_invalidos/consultas.txt

```
A,C
A,Z
solo_uno
A,A
```

Salida: problema1/salida_invalidos/errores_tramos.txt

```
# Errores en tramos.csv: línea | contenido | motivo
Línea 3 | B,C,x,2 | minutos debe ser un entero positivo y se recibió 'x'
Línea 4 | C,D | se esperaban 4 campos (origen,destino,minutos,costo) y se encontraron 2
```

Salida: problema1/salida_invalidos/itinerarios.txt

```
=== A → C ===
ERROR: el centro 'C' no existe en la red

=== A → Z ===
ERROR: el centro 'Z' no existe en la red

=== consulta inválida ===
ERROR: línea 3: consulta mal formada 'solo_uno' (se esperaba origen,destino)

=== A → A ===
Itinerario 1: A | tramos: 0 | 0 min | costo: 0
Total: 1 itinerario
Menos tramos: 1
Menor tiempo: 1
Menor costo: 1
```

Consola con carpeta inexistente

```
ERROR: falta el archivo requerido: carpeta_inexistente/tramos.csv
ERROR: falta el archivo requerido: carpeta_inexistente/consultas.txt
```

Casos límite y de tamaño

Entrada: problema1/datos_limite/consultas.txt

```
# D no tiene tramos de salida; A,A es el itinerario trivial
D,A
A,A
D,D
```

Salida: problema1/salida_limite/itinerarios.txt

```
=== D → A ===
sin itinerarios

=== A → A ===
Itinerario 1: A | tramos: 0 | 0 min | costo: 0
Total: 1 itinerario
Menos tramos: 1
Menor tiempo: 1
Menor costo: 1

=== D → D ===
Itinerario 1: D | tramos: 0 | 0 min | costo: 0
Total: 1 itinerario
Menos tramos: 1
Menor tiempo: 1
Menor costo: 1
```

Consola con archivos vacíos (datos_vacio)

```
Tramos bien formados: 0
Tramos descartados en la depuración: 0

Resumen por consulta:
```

Salida: problema1/salida_vacio/itinerarios.txt

```
(archivo vacío)
```

Entrada (primeras líneas): problema1/datos_grande/tramos.csv

```
# cadena de 10 rombos en serie: N0 -> ... -> N10
N0,P0,2,3
N0,Q0,5,1
P0,N1,4,2
Q0,N1,1,6
N1,P1,3,3
N1,Q1,5,2
P1,N2,4,2
Q1,N2,2,6
... (32 líneas omitidas en el informe; el archivo completo está en el repositorio) ...
```

Salida (recortada): problema1/salida_grande/itinerarios.txt

```
=== N0 → N10 ===
Itinerario 1: N0 → P0 → N1 → P1 → N2 → P2 → N3 → P3 → N4 → P4 → N5 → P5 → N6 → P6 → N7 → P7 →...
Itinerario 2: N0 → P0 → N1 → P1 → N2 → P2 → N3 → P3 → N4 → P4 → N5 → P5 → N6 → P6 → N7 → P7 →...
... (1021 líneas de itinerarios omitidas en el informe) ...
Itinerario 1024: N0 → Q0 → N1 → Q1 → N2 → Q2 → N3 → Q3 → N4 → Q4 → N5 → Q5 → N6 → Q6 → N7 → Q7 → N8 → Q8 → N9
    → Q9 → N10 | tramos: 20 | 69 min | costo: 115
Total: 1024 itinerarios
Menos tramos: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26,
    27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52,
    53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78,
    79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103,
    104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124,
    125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145,
    146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166,
    167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187,
    188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208,
    209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229,
    230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250,
    251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271,
    272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292,
... (18 líneas omitidas en el informe; el archivo completo está en el repositorio) ...
    671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689, 690, 691,
    692, 693, 694, 695, 696, 697, 698, 699, 700, 701, 702, 703, 704, 705, 706, 707, 708, 709, 710, 711, 712,
    713, 714, 715, 716, 717, 718, 719, 720, 721, 722, 723, 724, 725, 726, 727, 728, 729, 730, 731, 732, 733,
    734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754,
    755, 756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771, 772, 773, 774, 775,
    776, 777, 778, 779, 780, 781, 782, 783, 784, 785, 786, 787, 788, 789, 790, 791, 792, 793, 794, 795, 796,
    797, 798, 799, 800, 801, 802, 803, 804, 805, 806, 807, 808, 809, 810, 811, 812, 813, 814, 815, 816, 817,
    818, 819, 820, 821, 822, 823, 824, 825, 826, 827, 828, 829, 830, 831, 832, 833, 834, 835, 836, 837, 838,
    839, 840, 841, 842, 843, 844, 845, 846, 847, 848, 849, 850, 851, 852, 853, 854, 855, 856, 857, 858, 859,
    860, 861, 862, 863, 864, 865, 866, 867, 868, 869, 870, 871, 872, 873, 874, 875, 876, 877, 878, 879, 880,
    881, 882, 883, 884, 885, 886, 887, 888, 889, 890, 891, 892, 893, 894, 895, 896, 897, 898, 899, 900, 901,
    902, 903, 904, 905, 906, 907, 908, 909, 910, 911, 912, 913, 914, 915, 916, 917, 918, 919, 920, 921, 922,
    923, 924, 925, 926, 927, 928, 929, 930, 931, 932, 933, 934, 935, 936, 937, 938, 939, 940, 941, 942, 943,
    944, 945, 946, 947, 948, 949, 950, 951, 952, 953, 954, 955, 956, 957, 958, 959, 960, 961, 962, 963, 964,
    965, 966, 967, 968, 969, 970, 971, 972, 973, 974, 975, 976, 977, 978, 979, 980, 981, 982, 983, 984, 985,
    986, 987, 988, 989, 990, 991, 992, 993, 994, 995, 996, 997, 998, 999, 1000, 1001, 1002, 1003, 1004, 1005,
    1006, 1007, 1008, 1009, 1010, 1011, 1012, 1013, 1014, 1015, 1016, 1017, 1018, 1019, 1020, 1021, 1022,
    1023, 1024
Menor tiempo: 128, 256, 384, 512, 640, 768, 896, 1024
Menor costo: 1
```

Consola

```
Tramos bien formados: 40
Tramos descartados en la depuración: 0

Resumen por consulta:
  N0 → N10: 1024 itinerarios
```

2.2 Validación previa con recursión: depuración de la red

`Depuracion.depurar/1` recorre recursivamente los tramos válidos (recursión de cola con un mapa de parejas ya vistas como acumulador) y descarta los bucles y las repeticiones de la misma pareja en la misma dirección, conservando la primera aparición. Los tramos en sentido contrario coexisten. Cada descarte conserva la línea física del struct.

Fragmento: lib/depuracion.exs (sin Enum ni Stream)

```
  def depurar(tramos), do: depurar(tramos, %{}, [], [])

  defp depurar([], _vistos, aceptados, descartados) do
    {:lists.reverse(aceptados), :lists.reverse(descartados)}
  end

  defp depurar([%Tramo{origen: centro, destino: centro} = tramo | resto], vistos, aceptados,
    descartados) do
    motivo = "bucle: el tramo sale y llega al mismo centro (#{centro})"
    depurar(resto, vistos, aceptados, [{tramo, motivo} | descartados])
  end

  defp depurar([%Tramo{} = tramo | resto], vistos, aceptados, descartados) do
    pareja = {tramo.origen, tramo.destino}

    case Map.fetch(vistos, pareja) do
      {:ok, linea_primera} ->
        motivo = motivo_repetido(tramo, linea_primera)
        depurar(resto, vistos, aceptados, [{tramo, motivo} | descartados])

      :error ->
        depurar(resto, Map.put(vistos, pareja, tramo.linea), [tramo | aceptados], descartados)
    end
  end

  defp motivo_repetido(tramo, linea_primera) do
    "repetido: ya existe #{tramo.origen}→#{tramo.destino} (primera aparición en la línea
    #{linea_primera})"
  end
end
```

Caso	Tipo	Entrada	Resultado obtenido
Bucle A,A y repetido A,B,9,7	Válido	`datos/`	2 descartes: línea 8 (bucle) y línea 9 (repetido; primera aparición en la línea 2). B→C y C→B coexisten.
Formato incorrecto + descartes	Inválido	`datos_invalidos/`	Errores de formato en líneas 3 y 4 (no llegan a la depuración); descartes en líneas 5 (repetido) y 6 (bucle D,D).
Sin tramos	Límite	`datos_vacio/`	`depurar([])` devuelve `{[], []}`; archivos con solo encabezado.

Salida (válido): problema1/salida/depuracion_tramos.txt

```
# Tramos descartados: línea | contenido | motivo
Línea 8 | A,A,2,1 | bucle: el tramo sale y llega al mismo centro (A)
Línea 9 | A,B,9,7 | repetido: ya existe A→B (primera aparición en la línea 2)
```

Salida (válido): problema1/salida/errores_tramos.txt

```
# Errores en tramos.csv: línea | contenido | motivo
```

Salida (inválido): problema1/salida_invalidos/depuracion_tramos.txt

```
# Tramos descartados: línea | contenido | motivo
Línea 5 | A,B,9,7 | repetido: ya existe A→B (primera aparición en la línea 2)
Línea 6 | D,D,1,1 | bucle: el tramo sale y llega al mismo centro (D)
```

Salida (límite): problema1/salida_vacio/depuracion_tramos.txt

```
# Tramos descartados: línea | contenido | motivo
```

2.3 Stream infinito: centros alcanzables

`Alcance.pasos/2` construye con `Stream.iterate/2` la secuencia infinita S_0 = {origen}, $S_{n+1} = S_n \cup$ destinos de los tramos que salen de S_n. `Alcance.calcular/2` la consume con `Stream.transform/3` y corta (`:halt`) en el primer estado idéntico al anterior.

Fragmento: lib/alcance.exs

```
  def pasos(%Grafo{} = grafo, origen) do
    Stream.iterate(MapSet.new([origen]), fn alcanzados -> expandir(grafo, alcanzados) end)
  end

  def calcular(%Grafo{} = grafo, origen) do
    estados =
      grafo
      |> pasos(origen)
      |> Stream.transform(nil, &hasta_estabilizar/2)
      |> Enum.to_list()

    %{estados: estados, paso_estable: length(estados) - 1}
  end

  defp hasta_estabilizar(estado, estado), do: {:halt, estado}
  defp hasta_estabilizar(estado, _previo), do: {[estado], estado}
```

¿Por qué converge aunque el Stream sea infinito? (1) La secuencia es monótona: $S_n \subseteq S_{n+1}$, porque la unión nunca quita centros. (2) Está acotada: todos los estados son subconjuntos del conjunto finito de centros V. (3) Por tanto solo puede crecer estrictamente a lo sumo $|V|-1$ veces, y existe un n con $S_{n+1} = S_n$. (4) Como S_{n+1} depende únicamente de S_n, a partir de ese punto la secuencia es constante. El Stream no termina por sí mismo, pero el consumo es finito porque solo se piden estados hasta el primer repetido consecutivo.

Caso	Tipo	Entrada	Resultado obtenido
A→D y B→A	Válido	`datos/`	Desde A: pasos 0–2 y se estabiliza en el paso 2; D alcanzable. Desde B: se estabiliza en el paso 1; A no alcanzable → «sin itinerarios» sin búsqueda exhaustiva.
Centro inexistente	Inválido	`datos_invalidos/`	Para A→C y A→Z (centros inexistentes) no se calcula alcance; solo aparece el bloque de A→A.
D sin tramos de salida	Límite	`datos_limite/`	El Stream se estabiliza en el paso 0 (un único estado distinto).

Salida (válido): problema1/salida/alcance_centros.txt

```
=== A → D ===
Paso 0: {A}
Paso 1: {A, B, C}
Paso 2: {A, B, C, D}
Estados distintos: 3
Se estabiliza en el paso 2 (el paso 3 repite el paso 2)
El destino D es alcanzable

=== B → A ===
Paso 0: {B}
Paso 1: {B, C, D}
Estados distintos: 2
Se estabiliza en el paso 1 (el paso 2 repite el paso 1)
El destino A no es alcanzable: sin itinerarios (no se hizo búsqueda exhaustiva)
```

Salida (inválido): problema1/salida_invalidos/alcance_centros.txt

```
=== A → A ===
Paso 0: {A}
Paso 1: {A, B}
Estados distintos: 2
Se estabiliza en el paso 1 (el paso 2 repite el paso 1)
El destino A es alcanzable
```

Salida (límite): problema1/salida_limite/alcance_centros.txt

```
=== D → A ===
Paso 0: {D}
Estados distintos: 1
Se estabiliza en el paso 0 (el paso 1 repite el paso 0)
El destino A no es alcanzable: sin itinerarios (no se hizo búsqueda exhaustiva)

=== A → A ===
Paso 0: {A}
Paso 1: {A, B, C}
Paso 2: {A, B, C, D}
Estados distintos: 3
Se estabiliza en el paso 2 (el paso 3 repite el paso 2)
El destino A es alcanzable

=== D → D ===
Paso 0: {D}
Estados distintos: 1
Se estabiliza en el paso 0 (el paso 1 repite el paso 0)
El destino D es alcanzable
```

2.4 Pruebas automáticas del Problema 1

Se ejecutó `elixir pruebas.exs` (ExUnit). Resultado: **14 tests, 0 failures**.

Grupo	Prueba
Tramo.parse/1	válido
Tramo.parse/1	inválidos
Depuracion	descarta bucles y repetidos, conserva la primera y las direcciones contrarias
Depuracion	lista vacía
Busqueda y Criterios	ejemplo del enunciado
Busqueda y Criterios	sin camino y límite origen == destino
Busqueda y Criterios	no repite centros aunque haya ciclos
Busqueda y Criterios	empates en el mínimo se conservan
Alcance (Stream infinito)	se estabiliza y consume solo lo necesario
Alcance (Stream infinito)	el Stream es realmente infinito (se pueden pedir más pasos)
Alcance (Stream infinito)	origen sin salidas: estabiliza en el paso 0
Rutas.ejecutar/2 (extremo a extremo)	genera los archivos con el formato del enunciado
Rutas.ejecutar/2 (extremo a extremo)	falta de archivo: informa y no lanza excepción
Rutas.ejecutar/2 (extremo a extremo)	archivos vacíos (límite)

3. Problema 2 — Regiones en un mapa de colores y exportación SVG

El programa lee el mapa y la paleta por flujo, los valida recursivamente (si hay errores solo escribe `validacion_mapa.txt`), encuentra las regiones con dos funciones recursivas (`recorrer` y `explorar`), y genera `regiones.csv`, `resumen_regiones.txt` y el SVG por fragmentos.

3.1 Procesamiento principal: búsqueda de regiones

Fragmento: lib/buscador.exs (recorrer y explorar)

```
defp recorrer(mapa, {fila, columna} = posicion, visitadas, acumulado) do
  simbolo = Mapa.simbolo(mapa, posicion)
  siguiente = {fila, columna + 1}

  if simbolo == @fondo or MapSet.member?(visitadas, posicion) do
    recorrer(mapa, siguiente, visitadas, acumulado)
  else
    {celdas, visitadas} = explorar(mapa, posicion, simbolo, visitadas, [])
    region = construir(mapa, length(acumulado) + 1, simbolo, posicion, celdas)
    recorrer(mapa, siguiente, visitadas, [region | acumulado])
  end
end

defp explorar(mapa, posicion, simbolo, visitadas, celdas) do
  visitadas = MapSet.put(visitadas, posicion)
  explorar_vecinos(mapa, vecinos(posicion), simbolo, visitadas, [posicion | celdas])
end

defp explorar_vecinos(_mapa, [], _simbolo, visitadas, celdas), do: {celdas, visitadas}

defp explorar_vecinos(mapa, [vecino | resto], simbolo, visitadas, celdas) do
  if Mapa.simbolo(mapa, vecino) == simbolo and not MapSet.member?(visitadas, vecino) do
    {celdas, visitadas} = explorar(mapa, vecino, simbolo, visitadas, celdas)
    explorar_vecinos(mapa, resto, simbolo, visitadas, celdas)
  else
    explorar_vecinos(mapa, resto, simbolo, visitadas, celdas)
  end
end

defp vecinos({fila, columna}) do
  [{fila - 1, columna}, {fila + 1, columna}, {fila, columna - 1}, {fila, columna + 1}]
end

defp construir(mapa, id, simbolo, primera, celdas) do
```

Caso	Tipo	Entrada	Resultado obtenido
Ejemplo del enunciado	Válido	`datos/`	4 regiones; `regiones.csv` idéntico al del enunciado; mayor área y mayor perímetro: región 3 (C).
Mapa y paleta con 7 problemas	Inválido	`datos_invalidos/`	Solo se genera `validacion_mapa.txt`; no se calculan regiones ni SVG.
Archivos inexistentes	Inválido	`carpeta_inexistente/`	Se informa qué archivos faltan; sin excepción.
Solo fondo	Límite	`datos_limite_fondo/`	0 regiones; se genera igualmente el SVG con el fondo.
Una casilla	Límite	`datos_limite_1x1/`	1 región: área 1, perímetro 4.
Mapa 40×40	Tamaño	`datos_grande/`	Número de regiones: 605 · Mayor área (19): región 185 (B) · Mayor perímetro (40): región 185 (B)

Caso válido

Entrada: problema2/datos/mapa.txt

```
# mapa de ejemplo (símbolos separados por espacios)
A A . B B
A . . B .
. . C . .
C C C . B
C . . B B
```

Entrada: problema2/datos/paleta.csv

```
# simbolo,nombre,color_hex
A,agua,#2563EB
B,bosque,#16A34A
C,cultivo,#F59E0B
.,fondo,#F1F5F9
```

Salida: problema2/salida/regiones.csv

```
id;simbolo;area;perimetro;desde;hasta
1;A;3;8;{0,0};{1,1}
2;B;3;8;{0,3};{1,4}
3;C;5;12;{2,0};{4,2}
4;B;3;8;{3,3};{4,4}
```

Salida: problema2/salida/resumen_regiones.txt

```
Número de regiones: 4
Mayor área (5): región 3 (C)
Mayor perímetro (12): región 3 (C)
```

Consola

```
Validación correcta: mapa y paleta sin problemas.

Número de regiones: 4
Mayor área (5): región 3 (C)
Mayor perímetro (12): región 3 (C)

Archivos generados en salida/
```

Caso inválido

Archivos generados en problema2/salida_invalidos/ (no hay regiones.csv ni SVG)

```
validacion_mapa.txt
```

Consola con carpeta inexistente

```
ERROR: falta el archivo requerido: carpeta_inexistente/mapa.txt
ERROR: falta el archivo requerido: carpeta_inexistente/paleta.csv
```

Casos límite y de tamaño

Entrada: problema2/datos_limite_fondo/mapa.txt

```
. . .
. . .
```

Salida: problema2/salida_limite_fondo/regiones.csv

```
id;simbolo;area;perimetro;desde;hasta
```

Salida: problema2/salida_limite_fondo/resumen_regiones.txt

```
Número de regiones: 0
El mapa solo contiene fondo.
```

Entrada: problema2/datos_limite_1x1/mapa.txt

```
A
```

Salida: problema2/salida_limite_1x1/regiones.csv

```
id;simbolo;area;perimetro;desde;hasta
1;A;1;4;{0,0};{0,0}
```

Salida: problema2/salida_grande/resumen_regiones.txt (mapa 40×40)

```
Número de regiones: 605
Mayor área (19): región 185 (B)
Mayor perímetro (40): región 185 (B)
```

Salida (primeras líneas): problema2/salida_grande/regiones.csv

```
id;simbolo;area;perimetro;desde;hasta
1;B;3;8;{0,0};{1,1}
2;A;1;4;{0,1};{0,1}
3;C;1;4;{0,2};{0,2}
4;A;1;4;{0,3};{0,3}
5;B;2;6;{0,4};{0,5}
6;A;1;4;{0,6};{0,6}
7;B;1;4;{0,7};{0,7}
... (595 líneas omitidas en el informe; el archivo completo está en el repositorio) ...
603;B;2;6;{39,35};{39,36}
604;A;1;4;{39,37};{39,37}
605;A;1;4;{39,39};{39,39}
```

3.2 Validación previa con recursión: mapa y paleta

`Validacion.validar/2` verifica con funciones recursivas propias (sin `Enum` ni `Stream`) que las filas tengan igual longitud, que no haya filas vacías ni mapa sin filas, que todo símbolo esté en la paleta, que `.` exista, que los colores sean #RRGGBB y que no haya símbolos repetidos. Reporta **todos** los problemas (con línea, fila y columna) en `validacion_mapa.txt`.

Fragmento: lib/validacion.exs (verificación recursiva de longitudes)

```
  defp longitud_esperada([]), do: 0
  defp longitud_esperada([{_linea, simbolos} | _resto]), do: length(simbolos)

  defp longitudes([], _esperada, acumulado), do: :lists.reverse(acumulado)

  defp longitudes([{linea, []} | resto], esperada, acumulado) do
    longitudes(resto, esperada, ["mapa.txt línea #{linea}: la fila está vacía" | acumulado])
  end

  defp longitudes([{linea, simbolos} | resto], esperada, acumulado) do
    case length(simbolos) do
      ^esperada ->
        longitudes(resto, esperada, acumulado)

      otra ->
        problema = "mapa.txt línea #{linea}: tiene #{otra} columnas y se esperaban #{esperada}"
        longitudes(resto, esperada, [problema | acumulado])
    end
  end

  defp simbolos_indefinidos([], _fila, _definidos, acumulado), do: :lists.reverse(acumulado)

  defp simbolos_indefinidos([{linea, simbolos} | resto], fila, definidos, acumulado) do
    acumulado = indefinidos_en_fila(simbolos, linea, fila, 0, definidos, acumulado)
```

Caso	Tipo	Entrada	Resultado obtenido
Mapa y paleta del enunciado	Válido	`datos/`	«Validación correcta: sin problemas.»
Color 'azul', símbolo repetido, línea de 2 campos, falta '.', fila corta, símbolos 'Q' y '.' sin definir	Inválido	`datos_invalidos/`	7 problemas reportados, cada uno con línea (y fila/columna cuando aplica).
Mapa con solo comentarios	Límite	`datos_limite_mapa_vacio/`	«el mapa no contiene filas»; no se genera nada más.

Entrada (inválido): problema2/datos_invalidos/mapa.txt

```
A A Q
A .
```

Entrada (inválido): problema2/datos_invalidos/paleta.csv

```
A,agua,azul
A,otra,#000000
B,bosque
```

Salida (válido): problema2/salida/validacion_mapa.txt

```
Validación correcta: sin problemas.
```

Salida (inválido): problema2/salida_invalidos/validacion_mapa.txt

```
paleta.csv línea 1: el color 'azul' no tiene la forma #RRGGBB
paleta.csv línea 2: símbolo 'A' repetido (ya declarado en la línea 1)
paleta.csv línea 3: se esperaban 3 campos (simbolo,nombre,color_hex) y se encontraron 2
paleta.csv: no se define el símbolo '.' (fondo)
mapa.txt línea 2: tiene 2 columnas y se esperaban 3
mapa.txt línea 1 (fila 0, columna 2): el símbolo 'Q' no está definido en la paleta
mapa.txt línea 2 (fila 1, columna 1): el símbolo '.' no está definido en la paleta
```

Entrada (límite): problema2/datos_limite_mapa_vacio/mapa.txt

```
# mapa sin filas: solo comentarios y líneas en blanco
```

Salida (límite): problema2/salida_limite_mapa_vacio/validacion_mapa.txt

```
mapa.txt: el mapa no contiene filas
```

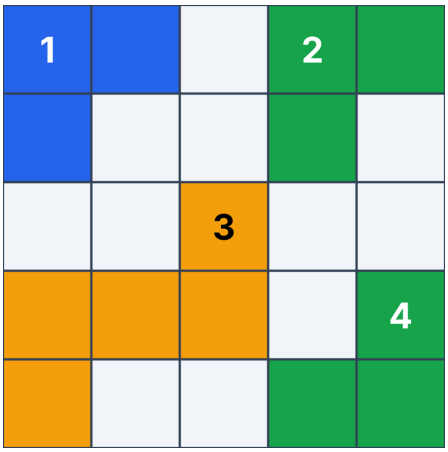
3.3 Generación del SVG

`Svg.fragmentos/3` devuelve un Stream perezoso: encabezado → un rectángulo de 40×40 por casilla (fondo incluido, con borde visible) → cierre. El ID de cada región se escribe en su primera casilla y todo texto que entra al XML pasa por `Svg.escapar/1`. El escritor consume el Stream fragmento a fragmento, sin armar el documento completo.

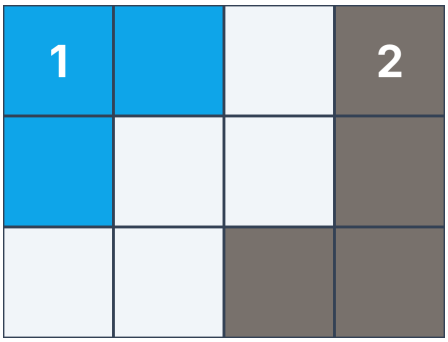
Carpeta de salida	Ancho × alto	<rect>	<text> (IDs)	XML bien formado	Dimensiones OK
salida	200 × 200 (esperado 200 × 200)	25	4	Sí	Sí
salida_limite_fondo	120 × 80 (esperado 120 × 80)	6	0	Sí	Sí
salida_limite_1x1	40 × 40 (esperado 40 × 40)	1	1	Sí	Sí
salida_escape	160 × 120 (esperado 160 × 120)	12	2	Sí	Sí
salida_grande	1600 × 1600 (esperado 1600 × 1600)	1600	605	Sí	Sí

Las verificaciones de la tabla se calcularon leyendo los SVG generados (parser XML `minidom`). Las imágenes siguientes son un render de esos mismos archivos; cada `mapa_regiones.svg` se abre con doble clic en el navegador.

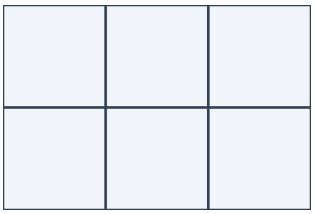
Caso	Tipo	Resultado obtenido
Mapa del enunciado 5×5	Válido	200 × 200 px, 25 rectángulos, 4 IDs; colores de la paleta y fondo visible.
Símbolos & y <, nombres con comillas	Válido (escape)	El XML sigue siendo válido: `&`, `<`, `"` en los `<title>`.
Paleta/mapa inválidos	Inválido	No se genera SVG (ver 3.2).
Solo fondo (2×3) y una casilla	Límite	SVG válido: 6 rectángulos sin IDs; 1 rectángulo con ID 1.



Mapa del enunciado (datos/)



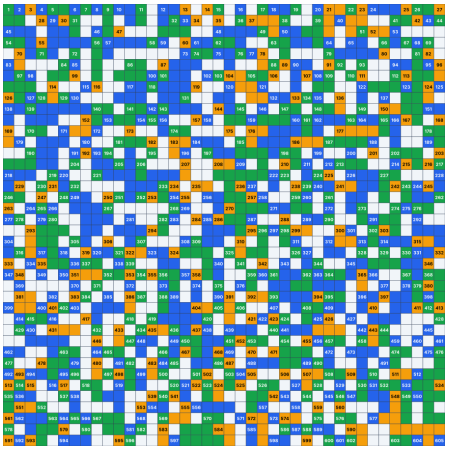
Caso de escape XML (datos_escape/)



Solo fondo



1×1



Mapa 40×40 (datos_grande/)

Fragmento: problema2/salida/mapa_regiones.svg (encabezado, primeras casillas y cierre)

```
<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" width="200" height="200" viewBox="0 0 200 200" font-family="sans-
    serif" font-size="16" text-anchor="middle">
<rect x="0" y="0" width="40" height="40" fill="#2563EB" stroke="#334155" stroke-width="1"><title>agua (A) en
     {0,0}</title></rect>
<text x="20" y="26" fill="#FFFFFF" font-weight="bold">1</text>
<rect x="40" y="0" width="40" height="40" fill="#2563EB" stroke="#334155" stroke-width="1"><title>agua (A)
    en {0,1}</title></rect>
...
<rect x="160" y="160" width="40" height="40" fill="#16A34A" stroke="#334155" stroke-width="1"><title>bosque
    (B) en {4,4}</title></rect>
</svg>
```

Fragmento: problema2/salida_escape/mapa_regiones.svg (texto escapado)

```
<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg" width="160" height="120" viewBox="0 0 160 120" font-family="sans-
    serif" font-size="16" text-anchor="middle">
<rect x="0" y="0" width="40" height="40" fill="#0EA5E9" stroke="#334155" stroke-width="1"><title>Mar &amp;
    &quot;Sal&quot; (&amp;) en {0,0}</title></rect>
<text x="20" y="26" fill="#FFFFFF" font-weight="bold">1</text>
<rect x="40" y="0" width="40" height="40" fill="#0EA5E9" stroke="#334155" stroke-width="1"><title>Mar &amp;
    &quot;Sal&quot; (&amp;) en {0,1}</title></rect>
<rect x="80" y="0" width="40" height="40" fill="#F1F5F9" stroke="#334155" stroke-width="1"><title>fondo (.)
    en {0,2}</title></rect>
```

Entrada: problema2/datos_escape/paleta.csv

```
&,Mar & "Sal",#0EA5E9
<,Roca <dura>,#78716C
.,fondo,#F1F5F9
```

Salida: problema2/salida_escape/regiones.csv

```
id;simbolo;area;perimetro;desde;hasta
1;&;3;8;{0,0};{1,1}
2;<;4;10;{0,2};{2,3}
```

3.4 Streams infinitos

El Problema 2 no pide un Stream infinito (aplica al Problema 1, sección 2.3). Sí usa Streams perezosos finitos: lectura de las líneas con `File.stream!/1`, transformaciones con `Stream.map/reject` y la generación del SVG casilla por casilla.

3.5 Pruebas automáticas del Problema 2

Se ejecutó `elixir pruebas.exs` (ExUnit). Resultado: **11 tests, 0 failures**.

Grupo	Prueba
Buscador	ejemplo del enunciado
Buscador	las diagonales no unen regiones
Buscador	solo fondo y una sola casilla (límites)
Buscador	mapa grande 300x300 sin desbordar (región en serpiente)
Validacion	mapa y paleta correctos
Validacion	detecta TODOS los problemas
Validacion	mapa sin filas
Svg	escapa texto peligroso
Mapas.ejecutar/2 (extremo a extremo)	entrada válida genera CSV, SVG y resumen
Mapas.ejecutar/2 (extremo a extremo)	entrada inválida: reporta y NO genera regiones ni SVG
Mapas.ejecutar/2 (extremo a extremo)	falta un archivo: informa sin excepción

4. Reflexión sobre el uso de inteligencia artificial

1. Herramientas usadas

Se usó el asistente conversacional **Claude** (Anthropic) mediante su interfaz de chat, en estas fases: lectura y resumen del enunciado; diseño de la arquitectura del código separando el «qué» del «cómo»; generación del código de los Problemas 1 y 2; creación de casos de prueba y de las pruebas ExUnit; y preparación del repositorio y del informe. No se usaron agentes ni extensiones de editor.

2. Interacción decisiva

Consulta: se pidió leer el PDF del parcial en detalle y generar el código de los problemas 1 y 2 «utilizando pensamiento computacional», separando el qué hacer del cómo hacerlo. **Respuesta:** una arquitectura con un módulo orquestador por problema (el qué: orden de los pasos) y módulos de algoritmo independientes (el cómo), más módulos de presentación y de entrada/salida. **Verificación y modificación:** el código se ejecutó y sus salidas se compararon con los ejemplos del enunciado (los 4 itinerarios y sus óptimos 1,3 / 1 / 3, el `regiones.csv` de 4 regiones). Al probar se detectó un error propio de la primera versión: las cláusulas de `Reporte` aceptaban `error: nil` como si fuera un error y el informe de consultas salía vacío. Se corrigió y luego el resultado de cada consulta se rediseñó con tuplas etiquetadas (`{:ok, ...}`, `{:sin_itinerarios, ...}`, `{:error, ...}`) para que ese tipo de error no pueda repetirse; las pruebas de extremo a extremo de `Rutas.ejecutar/2` verifican el texto de `itinerarios.txt`. Después se adaptó la entrega: de un proyecto Mix a scripts `.exs`, cada problema en su carpeta y cada módulo en su archivo, y por último el código se reescribió con la estructura de los ejemplos de clase (`main/0` como pipeline, funciones de varias cláusulas, atributos de módulo) y sin comentarios. Tras el rediseño se comprobó que los 42 archivos de salida de todos los casos quedaron idénticos a los anteriores.

3. Enfoques nuevos

Sugerencia: cortar un Stream infinito con `Stream.transform/3` devolviendo `{:halt, acumulador}` al detectar el primer estado repetido, y escribir las salidas con `Stream.into(File.stream!(ruta))`. **Qué aporta:** consume solo los pasos necesarios y escribe sin construir listas ni strings completos en memoria. **¿Se usó?** Sí, en `Alcance.calcular/2` y en `Escritor.volcar/2`.